

Hand an agent the chequebook. Keep the signature.

An AI agent that sources and negotiates real purchases, and a smart contract that will not let it spend a rupee more than a person authorised.

THE PROBLEM

Agents are trusted with judgement long before anyone trusts them with money.

Procurement is a good job for an agent. It is repetitive, it is quantitative, and the expertise is mostly in knowing what to compare.

It is also the job where a mistake costs money immediately, and nobody hands a spending limit to software they cannot constrain. So the agent drafts, and a person retypes the order into the system that actually pays.

Where the confidence breaks

- The limit lives in application code the agent runs inside
- A prompt can be talked around, and often is
- One compromised process can approve and pay
- The audit trail is written by the thing being audited

THE IDEA

Put the spending limit somewhere the agent cannot reach.

Not a guardrail

A guardrail is a check inside the program. It holds while the program behaves and it is the first thing to go when the program does not.

A boundary

The ceiling is contract state on a blockchain. The agent can ask to spend past it. The network refuses, and the refusal does not depend on our code being correct.

The interesting property is not that the agent behaves. It is that the system still holds when it does not.

One sentence in. One purchase out.

AGENT

Sources

Reads a request in plain language, pulls out quantity, budget, delivery window and certifications, and screens every listing against them.

AGENT

Negotiates

Bargains with shortlisted suppliers against reservation prices it cannot see, and walks away rather than break a hard requirement.

PEOPLE

Stops

Brings back one deal with its reasoning. Everything past this point needs a person, and the money needs three of them.

It is allowed to try. The contract is what stops it.

WHAT THE AGENT ATTEMPTED	AMOUNT	OUTCOME
Negotiated deal, inside the ceiling	\$1,175.00	SETTLED
Instructed to overspend by \$50	\$1,250.00	REVERTED ExceedsPerDealCap
Raised its own cap first, then tried	\$1,000,000.00	REVERTED buyer ceiling unchanged

The third row is the one that matters. The policy record is keyed to the address that wrote it, so an agent writing a policy can only ever write its own. Escalation is not a check that might be missed. It cannot be expressed.

No desk can do another desk's job.

Sales

Runs the sourcing, submits the purchase, confirms the goods arrived.

Cannot approve. Cannot pay.

Head

Publishes the spending ceiling on chain, approves or rejects, funds the escrow.

Cannot raise the request. Cannot pay.

Finance

Executes an authorised payment and nothing else.

Cannot raise. Cannot approve.

The role travels inside a signed token and is read back out of the signature, never out of anything a browser sends. Every restricted route checks it again on every request.

SETTLEMENT

A delivery takes two signatures.

- The supplier's key attests that the goods shipped
- The buyer's key confirms that they arrived
- The contract refuses the second without the first

Before this, one key could walk a deal from funded to paid with nothing having moved.

What this does not do

It does not make a fictitious delivery impossible. It makes it take two parties instead of one. A buyer and a supplier working together can still settle a deal that never moved.

Closing that needs an attestation from somebody with no stake in the trade, which means a carrier or an inspector. That is a partnership, not a function, and we would rather say so than imply we have solved it.

Two rails, kept apart on purpose.

The authority rail

An escrow contract holding USDC. It decides whether a payment is permitted, and it is what refuses a spend above the published ceiling.

It is not a bank account.

The money rail

Razorpay. A float is topped up through the payment gateway, and payouts to suppliers draw on it. It decides whether a permitted payment can actually be made.

Both have to say yes.

A payment is credited only after the server verifies the gateway's signature. The browser reporting success proves nothing, because a browser can be edited.

The model explains. It never computes.

Every figure in the product is produced by deterministic code: the screening, the bargaining, each amount, each ceiling.

A language model rewrites explanations so they read well. It is handed finished sentences and asked to phrase them better, and the result is checked digit by digit before it is shown.

The check

- Every number in the source must survive the rewrite
- Any number not in the source is refused
- If the rewrite fails either test, the deterministic text ships
- The interface says which one you are reading

LIM AI can read the run and explain it. It cannot approve a purchase, change a limit or move funds, and that boundary is the absence of an import rather than an instruction in a prompt.

HONESTY

What is real, and what is standing in.

REAL

Contracts, transactions, escrow, gas and reputation writes are genuine EVM execution

Role separation, the state machine and every refusal

Payment gateway, signature verification and payout idempotency

The screening and negotiation engine

STANDING IN FOR NOW

The chain runs in process by default. The same bytecode deploys to Base or any EVM network unchanged

Desk codes stand in for an identity provider. They stop the wrong tab becoming the approver, nothing more

Razorpay test mode. No real money moves

The supplier catalogue is seeded, and counterparties are simulated agents holding private floor prices

Every one of these is stated in the product itself, on the screen where it matters.

WHAT IS BUILT

Not a prototype of a demo.

280

unit and contract tests

- Postgres persistence with optimistic concurrency
- A buyer wallet per workspace, derived and isolated
- Deployment separated from boot, with bytecode drift detection

154

live API checks

81

checks in a real browser

- Audit trail with CSV and PDF export
- Supplier directory adapter for a real vendor list
- Notifications when work lands on a desk

Paid for the thing that is hard to build.

Who buys

Mid sized manufacturers and distributors with recurring input purchases, a small procurement team, and no appetite for an autonomous spender.

What they pay for

A platform fee on settled value. Nothing is charged when a deal does not complete, so the incentive is on purchases that actually close.

Why it holds

The supplier reputation record is written only by the escrow, only when funds move. It cannot be edited by us, by the buyer, or by the supplier, and any platform can read it.

The moat is not the model. It is an auditable record of who actually delivered, accumulated one settled purchase at a time.

NEXT

The three things that would change the answer.

ONE

Third party delivery attestation

A carrier or inspector with no stake in the trade. It is the last place where two willing parties can still agree on a fiction.

TWO

A real identity provider

Desk codes are a demonstration. Enterprise sign in is what makes the separation of duties an organisational fact rather than a product feature.

THREE

Live supplier directories

The adapter exists and the engine never cared about the catalogue's contents. Connecting a real vendor list is an integration, not a rewrite.

Watch it try to overspend, and watch it fail.

The demo runs a real chain in the page. Publish a ceiling, push the agent past it, and read the revert.

BUILT BY

Sujal Negi

Student, IIITDM Kurnool

sujalnegi.tech

github.com/sujal128005/limen